Booking API

One API, one key: a property's own website shows live availability and prices and books rooms, excursions and spa treatments directly into Uppsolut Stay.

What you can build

A website on the Booking API is a front for one property, or several properties of the same group behind one portal. Typically:

- **Property pages** — description, photos, location, contact details, room types and facilities.
- **A room booking page** — dates and guests, what is free and what it costs, minimal guest details, confirm.
- **An excursions page** — upcoming departures with seats left, then book for a party.
- **A spa page** — treatments by category, free start times on a day, then book, alone or as a couple.
- **A "my booking" page** — the live status of a booking, and self-cancellation where allowed.

Everything the site shows comes from Uppsolut Stay, so the property maintains one set of facts: the rates, capacity and closures the desk sets are exactly what the site sells, and a booking made on the site is at the desk immediately.

Modules and scopes

A key is enabled for one or more **modules**. Which ones a property sells online, and which of its excursions and treatments are published, is the property's choice in its administration area — your site reads it rather than hard-coding it.

Module	Scope	What it covers
Rooms	`ROOMS`	Availability calendar, stay quotes, room bookings, booking lookup.
Excursions	`EXCURSIONS`	Excursion catalogue, departures with live seats, quotes, holds, bookings.

Module	Scope	What it covers
Spa	`SPA`	Treatment catalogue, free start times, quotes, holds, bookings.
Transportation	`TRANSPORT`	The property's transfer operations — set-up, transfer bookings, departures, board, report, charges. Server-to-server only; not for a guest booking page.

`GET /properties/{id}` returns a `modules` block saying which modules your key can book at that property right now. Show a section only when it is enabled.

How a booking flows

1. **Browse.** Read the catalogue and live availability: nights for rooms, departures for excursions, start times for spa.
2. **Price.** Ask for a quote. It is the authoritative total, computed by the same code the property bills with — taxes, service charge and levies included. Never add prices up yourself.
3. **Hold (excursions and spa).** If your site takes payment before booking, hold the seats or the time slot first. The hold keeps them for a few minutes while the guest pays.
4. **Book.** Confirm with a fresh `Idempotency-Key`. The booking is instant: confirmed if the space is there, refused otherwise. Tell us whether the guest has paid.
5. **Confirm to the guest.** Show the reference and email it. Uppsolut Stay does not email guests for online bookings.
6. **Afterwards.** Look bookings up by reference and email, let guests cancel before the deadline, and receive webhooks when the property changes something.

> **NOTHING TO SYNCHRONISE**
>
> Your site holds no copy of availability or prices. Every call reads the live state of the property. Two guests racing for the last seat are resolved at booking time — the site is built to handle a `SOLD_OUT` answer gracefully and can never overbook.

Payment

The API takes no payment. Your site either takes payment itself (through your own provider) and sends the booking as `PAID` with your payment reference, or sends it as `UNPAID` for the guest to pay at the property. A paid booking is settled on the guest's bill automatically, so the desk sees it as paid.

Base URL

Every path in this guide is relative to `https://stay.uppsolut.com/api/website/v1` — use the address your property signs in at, with `/api/website/v1` after it.

Getting started →

Get a key and make your first calls.

Error codes →

Every code the API answers with.

Getting started

From a key to your first booking in a few calls.

1. Get a key from the property

The property's administrator creates your key in their administration area and ticks what it may use — Rooms, Excursions, Spa, Transportation — and whether it covers one property or all of the group's properties. They send you three values; keep them as **server-side secrets** (environment variables or a secrets manager), never in page source or a public repository:

```env
UPPSOLUT_API_BASE=https://stay.uppsolut.com/api/website/v1
UPPSOLUT_API_KEY=wsk_…
UPPSOLUT_PROPERTY_ID=…   # from GET /properties
```

Keys start with `wsk_`. The property can rotate or revoke a key at any time; a rotated key stops working at once.

2. List your properties

```shell
curl -H "Authorization: Bearer $UPPSOLUT_API_KEY" "$UPPSOLUT_API_BASE/properties"
```

This lists exactly the properties your key covers, with their ids.

3. Read the property and see what is live

```shell
curl -H "Authorization: Bearer $UPPSOLUT_API_KEY" "$UPPSOLUT_API_BASE/properties/$UPPSOLUT_P
```

json

```
{
  "property": {
    "id": "…",
    "name": "Coral Bay Resort",
    "businessDate": "2026-10-01",
    "…": "…",
    "modules": {
      "rooms":      { "enabled": true, "code": null, "reason": null },
      "excursions": { "enabled": true, "code": null, "reason": null },
      "spa":        { "enabled": false, "code": "NOT_SOLD_ONLINE", "reason": "The property d
    }
  }
}
```

Render a section of your site only when its module is `enabled`. Re-read this rather than hard-coding it: a property can switch a module on or off at any time and your site then follows without a code change.

4. A small client

javascript (node 18+)

```
const BASE = process.env.UPPSOLUT_API_BASE;
const KEY = process.env.UPPSOLUT_API_KEY;

export async function api(path, init = {}) {
  const res = await fetch(`${BASE}${path}`, {
    ...init,
    headers: { Authorization: `Bearer ${KEY}`, "Content-Type": "application/json", ...(init.
  });
  const body = await res.json();
  if (!res.ok) throw Object.assign(new Error(body.error), { code: body.code, status: res.sta
  return body;
}
```

5. Book an excursion, end to end

```javascript
const P = process.env.UPPSOLUT_PROPERTY_ID;

// What is on sale, and the departures in the next two weeks
const { excursions } = await api(`/properties/${P}/excursions`);
const { departures } = await api(`/properties/${P}/excursions/departures?from=2026-10-01&to=
const departure = departures.find((d) => d.bookable && d.seatsLeft >= 3);

// Hold the seats while the guest pays on your site
const { hold } = await api(`/properties/${P}/excursions/holds`, {
  method: "POST",
  body: JSON.stringify({ departureId: departure.id, adults: 2, children: 1 }),
});
// … take payment for hold.quote.totals.grandTotal with your own provider …

// Book — instant
const { booking } = await api(`/properties/${P}/excursions/bookings`, {
  method: "POST",
  headers: { "Idempotency-Key": crypto.randomUUID() },
  body: JSON.stringify({
    holdId: hold.holdId,
    guest: { firstName: "Ada", lastName: "Lovelace", email: "ada@example.com" },
    payment: { status: "PAID", provider: "YourProvider", reference: "pay_123", amount: hold.
    expectedTotal: hold.quote.totals.grandTotal,
  }),
});
console.log(booking.reference); // e.g. EXC-7K3QX9MD — show and email it to the guest
```

SERVER TO SERVER

Make these calls from your website's server, never from the visitor's browser. Excursion and spa bookings are refused from browser-enabled keys — see Authentication.

Authentication, limits and conventions

How to send your key, where it may live, how much you may call, and the conventions every endpoint follows.

Sending the key

Send the key on every request, in the `Authorization` header (preferred) or in `X-Api-Key`. Never in a URL.

```http
Authorization: Bearer wsk_…
```

Status	Code	Meaning
401	`MISSING_API_KEY`	No key header.
401	`INVALID_API_KEY`	Unknown, revoked or expired key — deliberately the same answer for all three.
403	`SCOPE_NOT_GRANTED`	The key is valid but not enabled for this module. Ask the property to tick it on your key.

Where the key lives

A key can create bookings in the property's system. Treat it like a database password.

Server to server (recommended)

Your website's backend calls the API and your pages call your backend. Any stack works — a Next.js or Nuxt server route, a PHP controller, a serverless function. The key never reaches a browser.

From the browser (static sites only)

The property can add your site's origin (e.g. `https://www.example.com`) to the key; the API then answers CORS for that origin. Anyone who reads your page source can then read the key.

> **BROWSER KEYS CANNOT BOOK EXCURSIONS OR SPA**
> An excursion or spa booking carries your site's statement that the guest has paid, so it must come from your server. A key with browser origins may read the excursion and spa catalogues and availability; holds, bookings and cancellations answer `403 SERVER_KEY_REQUIRED`.

Rate limits

Requests, per key	Limit per minute
`GET` — catalogues, availability, lookups	120
`POST` — quotes, holds, bookings, cancellations	20

Every response carries `RateLimit-Limit`, `RateLimit-Remaining` and `RateLimit-Reset` (seconds until the window resets). Past the limit you get `429 RATE_LIMITED` with `Retry-After`: wait that long and retry. A booking retried after a 429 must reuse its `Idempotency-Key`.

Requests with a missing or wrong key are counted per calling address: after 30 in a minute that address gets 429 instead of 401 until the window resets. Check the key in your configuration rather than retrying. A website may hold at most 50 live holds at a time (`429 TOO_MANY_HOLDS`).

To stay well inside the limits: search once per visitor search rather than once per day of a calendar, cache property details and catalogues for a few minutes, and never poll.

Conventions

JSON in, JSON out, UTF-8.

Dates are `YYYY-MM-DD` calendar days in the property's time zone; times are `HH:MM`, 24-hour, property local.

Money is a plain number in the property's currency.

Today depends on what you sell:

- **Rooms** — the property's `businessDate` (from `GET /properties/{id}`), which only moves when the desk closes the day. Build the stay picker off it, not the visitor's

clock.

Excursions and Spa — the actual date in the property's `timeZone`. Departures, free times and booking cutoffs run on the clock, not on the desk's day, and `businessDate` can be a day or more behind until the day is closed. Start activity pickers at whichever is later, today in the property's time zone or `businessDate`, and rely on `bookable` / `available` in the response rather than your own date arithmetic.

No caching: every response is `Cache-Control: no-store`. Availability is live.

Versioning: the `v1` path segment. Fields may be added to responses at any time; nothing is removed or renamed within v1. Ignore fields you do not know.

Errors are JSON with a stable `code` to switch on, a human `error`, and, for validation, `details` keyed by field. See Error codes.

```json
{ "error": "Invalid request.", "code": "VALIDATION", "details": { "guest.email": "A valid em
```

Idempotency

Every booking call needs an `Idempotency-Key` header: a fresh value (a UUID) per booking attempt. If a request times out or fails with a 5xx, retry with the **same** key: you get the same booking back (`200`, `replayed: true`), never a second one. Keys are scoped to your API key.

Rooms

Property pages, the availability calendar, stay quotes and room bookings. Needs the ROOMS scope, except for the two property endpoints.

List properties

`GET` `/properties` any scope

Every active property this key may act on.

```json
{
  "properties": [
    {
      "id": "8f1c…", "code": "CBR", "name": "Coral Bay Resort",
      "headline": "Barefoot luxury on a quiet lagoon",
      "currency": "USD", "timeZone": "Indian/Maldives", "starRating": 5,
      "imageUrl": "https://cdn.example.com/hero.jpg",
      "bookingEnabled": true
    }
  ]
}
```

Property details

`GET` `/properties/{propertyId}` any scope

Everything a property page needs. A property not on your key answers `404` `PROPERTY_NOT_FOUND` — never 403, so a key cannot be used to discover other properties.

json


```
{
  "property": {
    "id": "8f1c…", "code": "CBR", "name": "Coral Bay Resort", "legalName": "Coral Bay Hospit
    "headline": "Barefoot luxury on a quiet lagoon",
    "description": "Long-form copy set by the property…",
    "imageUrls": ["https://cdn.example.com/hero.jpg", "https://cdn.example.com/pool.jpg"],
    "logoUrl": "https://cdn.example.com/logo.png", "brandColor": "#0E6F7A", "starRating": 5,
    "address": "1 Lagoon Road",
    "location": { "latitude": 4.17, "longitude": 73.51 },
    "contact": { "phone": "+000 000 0000", "email": "stay@example.com" },
    "checkInTime": "14:00", "checkOutTime": "11:00",
    "currency": "USD", "timeZone": "Indian/Maldives", "pricesIncludeTaxes": true,
    "businessDate": "2026-10-01",
    "policies": "Free cancellation up to 7 days before arrival…",
    "facilities": [{ "name": "Infinity pool", "description": null }],
    "roomTypes": [
      {
        "id": "c2a1…", "code": "BVL", "name": "Beach Villa", "description": "…",
        "baseOccupancy": 2, "maxOccupancy": 3, "totalRooms": 12,
        "features": [
          { "category": "BED_TYPE", "code": "KING", "label": "King bed" },
          { "category": "ROOM_VIEW", "code": "OCEAN", "label": "Ocean view" }
        ]
      }
    ],
    "booking": {
      "enabled": true, "reason": null,
      "ratePlan": { "id": "…", "code": "BAR", "name": "Best Available Rate" },
      "mealPlanCode": "BB", "mealPlanSelectable": true, "mealPlanAffectsPrice": true,
      "mealPlans": [
        { "code": "RO", "name": "Room Only", "isDefault": false },
        { "code": "BB", "name": "Bed & Breakfast", "isDefault": true }
      ],
      "addOns": [
        { "id": "9a2f…", "code": "TRF-SB", "name": "Speedboat Transfer", "type": "TRANSFER",
      ],
      "minNights": 1, "maxNightsAhead": 365
    },
    "modules": {
      "rooms":      { "enabled": true, "code": null, "reason": null },
      "excursions": { "enabled": true, "code": null, "reason": null },
      "spa":        { "enabled": false, "code": "SCOPE_NOT_GRANTED", "reason": "This API key
    }
  }
}
```

`modules` — which modules this key can book here right now. `code` is one of `SCOPE_NOT_GRANTED` (ask the property to add it to your key), `ADDON_NOT_ENABLED`, `NOT_SOLD_ONLINE`, `NO_OUTLET`, `NOTHING_PUBLISHED` (the property is still setting it up) or, for rooms, `BOOKING_DISABLED`.

`roomTypes` lists only sellable types. `totalRooms` is informational — use the availability endpoint for what is free.

`booking.enabled` is false when the property has not chosen a rate plan or has switched online booking off; `booking.reason` says which. Show the property, hide the booking form.

`pricesIncludeTaxes` tells you how to label prices. The quote returns the tax breakdown either way.

Meal plans and extras are switched on per property, so both lists are often empty. `addOns` carry no price on purpose — what one costs depends on the party and the stay — send their ids to the quote and show the line it returns. Read the catalogue rather than hard-coding ids: a withdrawn extra is refused as `ADD_ON_NOT_FOUND`.

Availability calendar

GET `/properties/{propertyId}/availability?from=2026-10-10&to=2026-10-15` ROOMS

Per room type, per night in `[from, to)` — at most 62 nights per call.

json


```
{
  "propertyId": "8f1c…", "from": "2026-10-10", "to": "2026-10-15", "nights": 5,
  "currency": "USD", "pricesIncludeTaxes": true,
  "ratePlan": { "id": "…", "code": "BAR", "name": "Best Available Rate" },
  "bookingEnabled": true,
  "roomTypes": [
    {
      "id": "c2a1…", "code": "BVL", "name": "Beach Villa", "baseOccupancy": 2, "maxOccupancy
      "nights": [
        { "date": "2026-10-10", "available": 4, "closed": false, "price": 450, "extraAdultPr
        { "date": "2026-10-11", "available": 0, "closed": true,  "price": 450, "extraAdultPr
        { "date": "2026-10-12", "available": 2, "closed": false, "price": null, "extraAdultP
      ],
      "minAvailable": 0, "bookable": false, "roomRateTotal": null
    }
  ]
}
```

`available` — rooms of that type free that night. Real inventory: out-of-order rooms and group holds already excluded; never below 0.

`closed` — a stop-sale applies. Treat the night as unbookable.

`price` — the room rate for that night on the property's online rate plan, before tax handling. `null` means unpriced: the stay cannot be booked online.

`minAvailable`, `bookable`, `roomRateTotal` summarise the window.

> **DON'T ADD NIGHTLY PRICES UP**
> For the total a guest pays, call the quote. It adds taxes, service charge, levies and package components.

Stop sale

A stop sale is how a property closes dates to new bookings — a whole property, or one room type, for any number of nights. It applies to every rate plan and is set by the property, never through this API.

Where you see it	What it looks like
Availability	The night has `closed: true` and `available: 0`, whatever the real inventory. Other nights of the same stay are unaffected.
Quote	A stay that includes a closed night comes back with `available: false`. The prices are still shown; disable the booking button.
Booking	Refused with `409` and code `STOP_SALE`. If the type is also sold out, `STOP_SALE` is still the code you get.

A stop sale closes *nights*. Closing the departure date does not block a stay that leaves that day.

It never cancels or changes a booking that already exists, and it cannot be overridden through this API. Only the property can book a closed night, at its own desk.

It takes effect on the next call. Do not cache availability for longer than a few minutes, and always read `closed` rather than inferring it from `available`.

When the property lifts it, the nights return with their real `available` and `closed: false`. There is no webhook for this; read the availability again.

Where the property also sells through a channel manager, the same closure is sent to the channels at the same time, so the two stay in step.

Quote a stay

POST `/properties/{propertyId}/quote` ROOMS

```json
{ "checkIn": "2026-10-10", "checkOut": "2026-10-12", "roomTypeId": "c2a1…",
  "adults": 2, "children": 1, "mealPlanCode": "BB", "addOnIds": ["9a2f…"] }
```

The authoritative price, computed by the code the property bills with. Writes nothing.

```json
{
  "quote": {
    "propertyId": "8f1c…",
    "roomType": { "id": "c2a1…", "code": "BVL", "name": "Beach Villa" },
    "ratePlan": { "id": "…", "code": "BAR", "name": "Best Available Rate" },
    "mealPlanCode": "BB", "checkIn": "2026-10-10", "checkOut": "2026-10-12", "nights": 2,
    "adults": 2, "children": 1, "currency": "USD", "pricesIncludeTaxes": true,
    "available": true, "roomsAvailable": 4,
    "totals": { "roomBase": 900, "extraOccupancy": 80, "packageAllocations": 0, "taxes": 156
    "allocations": [
      { "id": "…", "code": "BF", "name": "Breakfast", "source": "MEAL_PLAN", "mode": "ADD_TO
    ],
    "taxLines": [{ "name": "TGST", "ratePercent": 16, "amount": 156.8 }],
    "nightly": [
      { "date": "2026-10-10", "rate": 450, "roomCharge": 490, "taxes": 87.4, "total": 577.4
    ],
    "warnings": []
  }
}
```

`available: false` — the stay cannot be booked right now (sold out, closed or unpriced). Show the price if you like, disable the button.

`totals.grandTotal` is the price to display.

`allocations` itemises the stay. `source` `RATE_PLAN` / `MEAL_PLAN` = included with the choice; `MANUAL` = an extra the guest ticked. `INCLUDE_IN_RATE` amounts are already inside `roomBase`.

`warnings` are for your logs, not for guests.

Create a booking

POST `/properties/{propertyId}/bookings` ROOMS · Idempotency-Key

```json
{
  "checkIn": "2026-10-10", "checkOut": "2026-10-12", "roomTypeId": "c2a1…",
  "adults": 2, "children": 1, "mealPlanCode": "BB", "addOnIds": ["9a2f…"],
  "guest": { "firstName": "Ada", "lastName": "Lovelace", "email": "ada@example.com", "phone"
  "remarks": "Arriving on the late seaplane"
}
```

Creates a real reservation (status `RESERVED`) with a confirmation number from the property's own sequence — exactly what the front desk would create. The guest is matched to an existing profile by email, or a new one is created. Response `201`, or `200` with `replayed: true` for a retry with the same key.

```json
{
  "booking": {
    "bookingId": "4c1e…", "confirmationNo": "CBR-000482", "reservationId": "…",
    "status": "RESERVED", "replayed": false,
    "property": { "id": "8f1c…", "name": "Coral Bay Resort", "checkInTime": "14:00", "checkO
    "guest": { "firstName": "Ada", "lastName": "Lovelace", "email": "ada@example.com" },
    "quote": { "…": "the confirmed figures, same shape as the quote" }
  }
}
```

Refusals (all 409): `SOLD_OUT`, `STOP_SALE`, `NO_RATE`, `BOOKING_DISABLED`. The site cannot overbook — send the guest back to the dates step. Room bookings take no payment; record any payment you take in `remarks`.

Look up a booking

GET `/bookings/{confirmationNo}?email=ada@example.com` ROOMS

Only bookings made with this key, and only with the email they were made with. Returns the booking with its live status: `RESERVED`, `IN_HOUSE`, `CHECKED_OUT`, `NO_SHOW` or `CANCELLED`. Room cancellations go through the property.

Excursions

Sell the property's excursions — boat trips, snorkelling, island hopping — with live seats, and book them instantly. Needs the EXCURSIONS scope.

The flow

1. **Catalogue** — what is sold online, today's prices, the rules (hold time, how far ahead booking closes, largest party).
2. **Departures** — the scheduled trips in a date range, with seats left.
3. **Quote** — the total for the party, taxes included.
4. **Hold** (optional, recommended if you take payment first) — keeps the seats for a few minutes.
5. **Book** — instant: confirmed if the seats are there, refused otherwise.

Catalogue

`GET` `/properties/{propertyId}/excursions`

```json
{
  "propertyId": "8f1c…", "currency": "USD", "pricesIncludeTaxes": true,
  "booking": {
    "enabled": true, "code": null, "reason": null,
    "holdMinutes": 10, "leadHours": 2, "maxPartySize": 10,
    "paidOnlineAccepted": true,
    "policies": "Free cancellation up to 24 hours before departure. Bring a towel and sun cr
  },
  "excursions": [
    {
      "id": "e71b…", "code": "DOLPH", "name": "Sunset Dolphin Cruise",
      "description": "Two hours along the reef at sunset…",
      "inclusions": "Soft drinks\nLife jackets",
      "imageUrls": ["https://cdn.example.com/dolphins.jpg"],
      "pricingMode": "PER_PERSON",
      "prices": { "adult": 85, "child": 45, "infant": 0 },
      "freeCancellationHours": 12
    },
    {
      "id": "a90c…", "code": "CHART", "name": "Private Fishing Charter",
      "pricingMode": "FLAT", "prices": { "flat": 650 }, "freeCancellationHours": 48, "…": "…
    }
  ]
}
```

Only excursions the property has published appear. The list can change at any time — read it, don't hard-code ids.

`pricingMode`: `PER_PERSON` prices each adult, child and infant; `FLAT` is one price for the whole party (a private charter).

`prices` are today's rate card before tax handling. A departure on a date with a different rate shows its own; the quote is the figure to charge.

`booking.enabled: false` means the property isn't taking online bookings right now (`reason` says why). You may still show the catalogue.

`paidOnlineAccepted: false` means the property can't yet receive bookings you've taken payment for — send them as `UNPAID`.

Departures

GET

`/properties/{propertyId}/excursions/departures?from=2026-10-01&to=2026-10-14[&excursionId=…]`

Scheduled departures in `[from, to]`, both inclusive, at most 62 days per call. Departures that have left, are cancelled, or belong to unpublished excursions are left out.

```json
{
  "propertyId": "8f1c…", "from": "2026-10-01", "to": "2026-10-14", "currency": "USD", "booki
  "departures": [
    {
      "id": "d4f2…", "excursionId": "e71b…", "date": "2026-10-03", "time": "17:00",
      "meetingTime": "16:45", "meetingPoint": "Sunset Jetty",
      "capacity": 20, "seatsLeft": 7, "minGuests": 6, "guaranteed": true,
      "bookingClosesAt": "2026-10-03T13:00:00.000Z",
      "prices": { "adult": 85, "child": 45, "infant": 0 },
      "bookable": true
    }
  ]
}
```

`seatsLeft` is live: confirmed bookings and other websites' holds are already taken off. Infants take a seat too.

`guaranteed: false` — the departure has fewer guests than its minimum and the property may still call it off. Say so on your page.

`bookingClosesAt` — online booking closes this long before departure (the property's "book at least … hours ahead").

`bookable` — seats left, still open, priced, and the property is taking online bookings.

Departures run on the clock: ask from today in the property's time zone, not from `businessDate`, which can lag until the desk closes the day. See Conventions.

Quote

POST `/properties/{propertyId}/excursions/quote`

```json
{ "departureId": "d4f2…", "adults": 2, "children": 1, "infants": 0 }
```

```json
{
  "quote": {
    "departureId": "d4f2…", "excursionId": "e71b…",
    "adults": 2, "children": 1, "infants": 0,
    "currency": "USD", "pricesIncludeTaxes": true,
    "totals": { "base": 184.13, "serviceCharge": 18.41, "taxes": 12.46, "levies": 0, "grandT
    "lines": [
      { "description": "Sunset Dolphin Cruise — 2 adults, 1 child (2026-10-03 17:00)", "amou
    ],
    "available": true, "seatsLeft": 7
  }
}
```

The quote is computed by the same code that posts the charge to the guest's bill, so the booking posts exactly `totals.grandTotal`. It writes nothing. `available: false` means it can't be booked right now (not enough seats, closed, or cut off) — show the price if you like, but disable the button.

Hold

POST `/properties/{propertyId}/excursions/holds` server-only key

```json
{ "departureId": "d4f2…", "adults": 2, "children": 1 }
```

```json
{
  "hold": {
    "holdId": "0b7e…",
    "expiresAt": "2026-10-01T09:12:00.000Z",
    "quote": { "…": "same as the quote endpoint" }
  }
}
```

The seats are yours until `expiresAt` (the property's hold time, typically 10 minutes) — nobody else, desk or website, can take them. Take payment, then book with the `holdId`. An unused hold simply expires; there is nothing to release. Refusals: `SOLD_OUT`, `DEPARTURE_CLOSED`, `BOOKING_CUTOFF`, `PARTY_TOO_LARGE`, `TOO_MANY_HOLDS`.

Book

POST `/properties/{propertyId}/excursions/bookings` server-only key · Idempotency-Key required

From a hold

```json
{
  "holdId": "0b7e…",
  "guest": { "firstName": "Ada", "lastName": "Lovelace", "email": "ada@example.com", "phone"
  "payment": { "status": "PAID", "provider": "YourProvider", "reference": "pay_123", "amount
  "expectedTotal": 215,
  "remarks": "One guest is a nervous swimmer"
}
```

Directly

```json
{
  "departureId": "d4f2…", "adults": 2, "children": 1, "infants": 0,
  "guest": { "firstName": "Ada", "email": "ada@example.com" },
  "payment": { "status": "UNPAID" }
}
```

Field	Notes
`guest`	firstName and email are required. The email is how the guest finds the booking again.
`payment.status`	`PAID` — you took payment; the booking is settled on the bill with the property's online payment method. `UNPAID` — the guest pays at the property.
`payment.reference`	Your payment reference. Shown to the desk; use it for refunds.
`payment.amount`	What you charged. If it differs from the total the booking still goes through and the desk is told to check.
`expectedTotal`	The `grandTotal` you showed. If the price moved since, the booking is refused with `PRICE_CHANGED` and nothing is created.
`remarks`	Anything the guest wants the property to know. Up to 1000 characters.

Response `201 Created` — or `200` with `replayed: true` when you retried with the same Idempotency-Key:

```json
{
  "booking": {
    "reference": "EXC-7K3QX9MD",
    "module": "EXCURSIONS",
    "status": "CONFIRMED",
    "replayed": false,
    "property": { "id": "8f1c…", "name": "Coral Bay Resort" },
    "guest": { "firstName": "Ada", "lastName": "Lovelace", "email": "ada@example.com" },
    "payment": { "status": "PAID", "reference": "pay_123" },
    "total": { "grandTotal": 215, "currency": "USD" },
    "moved": false,
    "excursion": { "id": "e71b…", "name": "Sunset Dolphin Cruise" },
    "departure": { "id": "d4f2…", "date": "2026-10-03", "time": "17:00", "meetingTime": "16:
    "adults": 2, "children": 1, "infants": 0,
    "cancellation": { "allowed": true, "freeUntil": "2026-10-03T05:00:00.000Z", "cancelledAt
  }
}
```

Show the `reference` on your confirmation page and in your email to the guest, with the date, time and meeting point.

Refusals

Code	What to do
`SOLD_OUT`	Not enough seats left (details.seatsLeft). Send the guest back to choose another departure.
`HOLD_EXPIRED`	The hold ran out. Check the departure again and hold/book afresh.
`HOLD_USED`	This hold was already booked — look the booking up instead of booking twice.
`PRICE_CHANGED`	Re-quote, show the new total, confirm with the guest. Refund any difference you took.
`BOOKING_CUTOFF`	Online booking for this departure has closed. The guest can contact the property.
`DEPARTURE_CLOSED`	Cancelled or already left.

Code	What to do
`PAYMENT_NOT_CONFIGURED`	The property can't receive paid bookings yet — send UNPAID or ask them to set it up.
`PARTY_TOO_LARGE`	More guests than the property allows per online booking.
`MODULE_NOT_ENABLED`	The property isn't selling excursions online right now.

> **IF YOU TOOK PAYMENT AND THE BOOKING IS REFUSED**
>
> Refund the guest through your provider. Holding first makes this rare: a hold guarantees the seats while the guest pays.

What happens at the property

The booking appears on the departure's manifest immediately, marked *Online*with its reference, on a bill in the guest's name. If the property later cancels the departure (weather), moves the guest to another departure, or marks a no-show, the lookup shows it and — if you registered one — a webhook tells you.

Spa

Sell the property's spa treatments with live free times — for one guest or a couple — and book them instantly. Needs the SPA scope.

The flow

1. **Treatments** — what is sold online, by category, with duration and price.
2. **Free times** — for a treatment and party size: which days have any time free, and the start times on a day.
3. **Quote** — the total, taxes included.
4. **Hold** (optional, recommended if you take payment first) — keeps a therapist per guest and a room.
5. **Book** — instant: confirmed if the time is still free, refused otherwise.

> **THERAPISTS AND ROOMS ARE ASSIGNED FOR YOU**
>
> The property's own scheduling engine picks a qualified, free therapist for each guest and a suitable free room, exactly as it does at the spa desk. Therapists and rooms are never named in the API. Where the property allows it, a guest may ask for a female or male therapist.

Treatments

`GET` `/properties/{propertyId}/spa/treatments`

```json
{
  "propertyId": "8f1c…", "currency": "USD", "pricesIncludeTaxes": true,
  "booking": {
    "enabled": true, "code": null, "reason": null,
    "holdMinutes": 10, "leadHours": 2,
    "genderPreferenceOffered": true,
    "freeCancellationHours": 4,
    "paidOnlineAccepted": true,
    "policies": "Please arrive 15 minutes early…"
  },
  "categories": [
    {
      "name": "Massage",
      "treatments": [
        {
          "id": "t19a…", "name": "Aromatherapy Massage",
          "description": "A slow, relaxing full-body massage…",
          "inclusions": "Herbal tea", "imageUrls": ["https://cdn.example.com/aroma.jpg"],
          "durationMinutes": 60, "maxGuests": 1, "pricingMode": "PER_PERSON", "price": 130
        }
      ]
    },
    {
      "name": "Couples",
      "treatments": [
        { "id": "t77c…", "name": "Couples Sunset Ritual", "durationMinutes": 90, "maxGuests"
      ]
    }
  ]
}
```

`pricingMode` : `PER_PERSON` — `price` is per guest; `FLAT` — one price for the whole party.

`maxGuests` — how many guests one booking of this treatment can carry (2 for a couples treatment).

`freeCancellationHours` — guests may cancel online until this long before the start.

Free times

Start times on a day

GET

`/properties/{propertyId}/spa/availability?treatmentId=t19a…&date=2026-10-03&partySize=1[&gender=FEMALE]`

```json
{
  "treatmentId": "t19a…", "date": "2026-10-03", "partySize": 1, "durationMinutes": 60,
  "slots": [
    { "startTime": "09:00", "endTime": "10:00", "available": false },
    { "startTime": "09:30", "endTime": "10:30", "available": true },
    { "startTime": "10:00", "endTime": "11:00", "available": true }
  ]
}
```

Which days have any time

GET

`/properties/{propertyId}/spa/availability?treatmentId=t19a…&from=2026-10-01&to=2026-10-31&partySize=2`

```json
{ "treatmentId": "t19a…", "partySize": 2, "days": [{ "date": "2026-10-01", "available": true
```

> At most 31 days per range call. Use the range to grey out full days in a date picker, then the day call for times.
>
> Start the picker at today in the property's time zone, not at `businessDate` — free times run on the clock, and `businessDate` can lag until the desk closes the day. See Conventions.
>
> A slot is available when there is a free therapist for **every** guest and a free room for the party, and it is not inside the property's "book at least … hours ahead" window.
>
> `gender` (`MALE` or `FEMALE`) filters to therapists of that gender for every guest. Only when `genderPreferenceOffered` is true; otherwise it is refused.
>
> Holds by other websites are taken into account — a held time shows as not available.

Quote

POST `/properties/{propertyId}/spa/quote`

```json
{ "treatmentId": "t19a…", "date": "2026-10-03", "partySize": 1 }
```

```json
{
  "quote": {
    "treatmentId": "t19a…", "date": "2026-10-03", "partySize": 1,
    "currency": "USD", "pricesIncludeTaxes": true,
    "totals": { "base": 111.36, "serviceCharge": 11.14, "taxes": 7.5, "levies": 0, "grandTot
    "lines": [{ "description": "Aromatherapy Massage", "amount": 130 }]
  }
}
```

Hold

POST `/properties/{propertyId}/spa/holds` server-only key

```json
{ "treatmentId": "t19a…", "date": "2026-10-03", "startTime": "09:30", "partySize": 1, "gende
```

```json
{
  "hold": {
    "holdId": "h3d1…", "expiresAt": "2026-10-01T09:12:00.000Z", "startTime": "09:30",
    "quote": { "…": "same as the quote endpoint" }
  }
}
```

A therapist per guest and a room are reserved until `expiresAt`; the spa desk sees the time as "held online". Then book with the `holdId`. An unused hold expires on its own. Refusals: `SLOT_UNAVAILABLE`, `BOOKING_CUTOFF`, `PARTY_TOO_LARGE`, `TOO_MANY_HOLDS`.

Book

POST `/properties/{propertyId}/spa/bookings` server-only key · Idempotency-Key required

From a hold

```json
{
  "holdId": "h3d1…",
  "guest": { "firstName": "Grace", "lastName": "Hopper", "email": "grace@example.com" },
  "payment": { "status": "PAID", "provider": "YourProvider", "reference": "pay_456", "amount
  "expectedTotal": 130
}
```

Directly, for a couple

```json
{
  "treatmentId": "t77c…", "date": "2026-10-03", "startTime": "16:00", "partySize": 2,
  "guest": { "firstName": "Grace", "lastName": "Hopper", "email": "grace@example.com" },
  "companions": ["Alan Turing"],
  "payment": { "status": "UNPAID" }
}
```

Field	Notes
`guest`	The booking guest (the first participant). firstName and email are required.
`companions`	Names of the other guests, in order. Optional — unnamed guests appear as Guest 2, Guest 3.
`gender`	Optional therapist gender for every guest, where offered. Ignored when booking from a hold (the hold already has its therapists).
`payment`	Same as excursions: `PAID` settles the bill with the property's online payment method; `UNPAID` is paid at the spa.
`expectedTotal`	Checked against what is actually charged. A mismatch refuses the booking with `PRICE_CHANGED` and nothing is created.

Response `201` (or `200` with `replayed: true`):

```json
{
  "booking": {
    "reference": "SPA-M2P8RD4K", "module": "SPA", "status": "CONFIRMED", "replayed": false,
    "property": { "id": "8f1c…", "name": "Coral Bay Resort" },
    "guest": { "firstName": "Grace", "lastName": "Hopper", "email": "grace@example.com" },
    "payment": { "status": "UNPAID", "reference": null },
    "total": { "grandTotal": 340, "currency": "USD" },
    "treatment": { "id": "t77c…", "name": "Couples Sunset Ritual" },
    "date": "2026-10-03", "startTime": "16:00", "endTime": "17:30",
    "partySize": 2, "guests": ["Grace Hopper", "Alan Turing"],
    "cancellation": { "allowed": true, "freeUntil": "2026-10-03T08:00:00.000Z", "cancelledAt
  }
}
```

Refusals

Code	What to do
`SLOT_UNAVAILABLE`	The time was taken meanwhile. Show the day's free times again.
`HOLD_EXPIRED`	The hold ran out. Check free times and hold/book afresh.
`HOLD_USED`	Already booked — look it up rather than booking twice.
`PRICE_CHANGED`	Re-quote and confirm the new total with the guest.
`BOOKING_CUTOFF`	Too close to the start to book online.
`PARTY_TOO_LARGE`	More guests than the treatment allows.
`PAYMENT_NOT_CONFIGURED`	Send UNPAID, or ask the property to set up paid online bookings.

Transportation

Run a property's guest transfers from your own system — the transport set-up, transfer bookings, shared departures, the daily board and report, and charging transfers to the guest's bill. Needs the TRANSPORT scope and a server-only key.

> **NOT FOR A PUBLIC BOOKING PAGE**
>
> This module is for the property's operations — an operator's dispatch system, the property's own tools, a reporting job. It carries guest names, flights and contacts and can post charges, so a key with the `TRANSPORT` scope must be used from a server: keys with browser origins can't be given the scope, and are refused with `403 SERVER_KEY_REQUIRED`. Guests booking a transfer on a website is not part of this API.

Before you start

Transportation is an add-on: it must be enabled for the enterprise first. Without it the Transportation scope can't be given to a key, and every endpoint here answers `409 MODULE_NOT_ENABLED` with `details.reason` `ADDON_NOT_ENABLED`.

The property ticks **Transportation** on your key in the Hub (Booking API keys). No browser origins on that key.

The property switches Transportation on (Hub → the property → Transportation). Until then the configuration endpoints work, the rest answer `409 MODULE_NOT_ENABLED` with `details.reason` `NOT_ENABLED`.

Everything lives under `/properties/{propertyId}/transport`. A property your key doesn't cover is `404 PROPERTY_NOT_FOUND`.

Times, dates and money

Local times you send and read (`departureTime`, `flightTime`) are `HH:MM`, 24-hour, in the property's time zone (`timeZone` below — `Indian/Maldives` for Maldivian properties).

Days (`serviceDate`, `flightDate`, `from`, `to`) are `YYYY-MM-DD`, the property's local day.

Instants in responses (`flightAt`, `departureAt`) are UTC ISO 8601; each comes with a `…Local` twin (`{ dateKey, time }`) so you never convert.

Amounts are in the property's currency and entered the way the property enters every price — including or excluding tax per `pricesIncludeTaxes`. The posted folio total (tax included) is returned when a charge posts.

`GET` `/properties/{propertyId}/transport`

```json
{ "propertyId": "8f1c…", "enabled": true, "timeZone": "Indian/Maldives", "currency": "USD",
  "pricesIncludeTaxes": true, "attentionToleranceMinutes": 60, "requireProvider": false }
```

Configuration

The same catalogue the property edits in the Hub, with the same rules. Six collections:

Collection	What it is	Key fields
`types`	Speedboat, seaplane, domestic flight, ferry, land	code, name, mode, requiresFlightDetails
`locations`	Airports, jetties, seaplane platforms, islands, resorts, guesthouses	code, name, type, notes
`routes`	From → to, by a transport type, with default departure times	code, name, originId, destinationId, transportTypeId, category, direction, durationMinutes, departureSlots
`providers`	The property's own fleet or a third-party operator	name, kind (OWN | THIRD_PARTY), contactName, phone, email
`vessels`	Boats, seaplanes, vehicles, with seats	providerId, name, transportTypeId, capacity, registration
`rates`	Prices per route	routeId, direction, pricingBasis, price | adultPrice/childPrice/infantPrice, childMinAge, childMaxAge, validFrom, validTo, chargeCodeId, taxMode, taxProfileId, isBillable

`GET` `/properties/{propertyId}/transport/config/{collection}[?active=1]`

`POST` `/properties/{propertyId}/transport/config/{collection}`

`GET` `/properties/{propertyId}/transport/config/{collection}/{id}`

`PATCH` `/properties/{propertyId}/transport/config/{collection}/{id}`

partial; { isActive: false } deactivates

`DELETE` `/properties/{propertyId}/transport/config/{collection}/{id}` only while nothing uses it

> Every row has `isActive`. Anything already used by a booking, rate or departure can't be deleted (`409 IN_USE`) — deactivate it.
>
> Codes are unique per property (`409 DUPLICATE`); ids you reference must be this property's own (`400 INVALID_REFERENCE`).
>
> A route used for `BOTH` directions is entered the arrival way round (airport → property); a drop-off travels it in reverse — bookings return the real `route.from` and `route.to`.
>
> `pricingBasis` : `PER_PERSON` (adult/child/infant prices), `PER_VEHICLE` (`price` × vehicles), `PER_TRIP` (`price` per booking). `taxMode` : `CHARGE_CODE` , `DEFAULT` (Service Charge + GST) or `CUSTOM` (with `taxProfileId`). A route without a rate is complimentary.

Bookings

One guest party, one direction: a `PICKUP` (arrival) or a `DROP_OFF` (departure). Linked to a reservation, or a traveller with no stay (billed on a walk-in bill).

`POST` `/properties/{propertyId}/transport/bookings`

```json
{
  "reservationId": "c3d9…",          // or, with no stay: "guestName": "…", "guestContact":
  "direction": "PICKUP",
  "serviceDate": "2026-12-20",       // defaults to the reservation's arrival / departure da
  "adults": 2, "children": 1, "infants": 0,   // default to the reservation's party
  "routeId": "r81a…", "departureTime": "14:00",
  "airline": "Emirates", "flightNo": "EK652", "flightTime": "12:15", "terminal": "Internatio
  "airportRepUserId": null, "meetingNotes": "Counter 12, 4 bags",
  "providerId": null, "vesselId": null, "driverName": null, "driverContact": null, "seatNote
  "notes": null,
  "status": "CONFIRMED"              // or "DRAFT"
}
```

The price comes from the route's most specific enabled rate. Send `"priceOverride": { "amount": 80, "reason": "…" }` to set it by hand.

```json
{
  "id": "b40355…", "reference": "TR-B40355", "status": "CONFIRMED", "direction": "PICKUP",
  "serviceDate": "2026-12-20", "adults": 2, "children": 1, "infants": 0, "pax": 3,
  "guestName": "Aisha Rasheed", "reservation": { "id": "c3d9…", "confirmationNo": "VL4201",
  "groupBlock": { "id": "…", "code": "WED24", "name": "Wedding party" },
  "flightNo": "EK652", "flightAt": "2026-12-20T07:15:00.000Z", "flightLocal": { "dateKey": "
  "route": { "id": "r81a…", "code": "MLE-SPB", "name": "Airport – resort by speedboat",
            "from": { "code": "MLE", "…": "…" }, "to": { "code": "HOME", "…": "…" }, "…": "
  "effectiveDepartureAt": "2026-12-20T09:00:00.000Z", "departureLocal": { "dateKey": "2026-1
  "manifest": null, "provider": null, "vessel": null, "isOwn": null,
  "pricing": { "rateId": "…", "amount": 300, "priceOverridden": false, "chargeCodeId": "…",
  "billing": { "status": "NOT_BILLED", "folioLineItemId": null, "postedGross": null, "note":
  "attention": [{ "code": "TOO_SOON_AFTER_LANDING", "message": "Departs 105 min after landin
}
```

GET

`/properties/{propertyId}/transport/bookings?from=&to=&direction=&status=&reservationId=&manifestId=&q=&unassigned=1&attention=1&limit=&cursor=`

Paginated: `{ "data": [...], "nextCursor": "…" | null }`. `limit` is 1–200 (default 100); pass `nextCursor` back as `cursor` for the next page. Cancelled bookings are left out unless asked for by status.

GET `/properties/{propertyId}/transport/bookings/{bookingId}`

PATCH `/properties/{propertyId}/transport/bookings/{bookingId}`

any field above; re-prices an unposted transfer

POST `/properties/{propertyId}/transport/bookings/{bookingId}/status`

{ "status": "NO_SHOW", "reason": "..." }

Statuses: `DRAFT` → `CONFIRMED` → `ASSIGNED` (on a departure or with a provider) → `COMPLETED`; or `NO_SHOW` / `CANCELLED`. A cancelled booking leaves its departure. The response carries a `note` when a charge needs a person's attention.

POST `/properties/{propertyId}/transport/quote`

the rate and folio total for a booking you are about to make

Needs attention

Worked out live, never stored, and never a reason a save is refused: `TOO_SOON_AFTER_LANDING`, `TOO_CLOSE_TO_FLIGHT` (the route's duration counts), `FLIGHT_CHANGED` (since the booking joined its departure — clear it with `KEEP` below) and `FLIGHT_MISSING`, and `RESERVATION_CLOSED` (the stay was cancelled or never arrived — Night Audit won't charge the transfer; cancel it or mark it a no-show). The allowance is the property's `attentionToleranceMinutes`.

Departures (manifests)

One boat, seaplane or car leaving at a time; bookings from any reservations ride on it together. Seats are counted live; more passengers than the vessel's seats is reported (`capacityState: "OVER"`) but never refused.

GET `/properties/{propertyId}/transport/manifests?from=&to=&direction=&routeId=`

POST `/properties/{propertyId}/transport/manifests`

routeId, direction, serviceDate, departureTime, providerId?, vesselId?, driverName?, bookingIds?

GET `/properties/{propertyId}/transport/manifests/{manifestId}`

PATCH `/properties/{propertyId}/transport/manifests/{manifestId}`

time, vessel, driver, notes, status

POST `/properties/{propertyId}/transport/manifests/{manifestId}/bookings`

{ "action": "ATTACH" | "DETACH" | "KEEP", "bookingIds": [...] }

POST `/properties/{propertyId}/transport/manifests/from-slots`

the day's departures from the routes' default times

> `ATTACH` also moves a booking from another departure, and puts it on the departure's day. A confirmed booking becomes `ASSIGNED`; a draft stays a draft.
>
> Status `COMPLETED` completes its passengers; `CANCELLED` takes them off, still booked, to be put on another departure.

Board, report and suggestions

GET `/properties/{propertyId}/transport/board?date=YYYY-MM-DD[&filters]`

the day's bookings, departures and a week of counts

GET `/properties/{propertyId}/transport/report?from=&to=`

the Daily Transportation Report's data, ≤ 62 days

`GET` `/properties/{propertyId}/transport/suggestions?from=&to=`

arrivals and departures with nothing booked — nothing is created

Charging transfers

Billable transfers are posted to the guest's bill by the property's Night Audit — pickups on the arrival day, drop-offs on the guest's last night (dated the departure day) — once per booking, with the rate's charge code and tax. Drafts, no-shows and cancellations are never posted automatically. To post or waive by hand:

`POST` `/properties/{propertyId}/transport/bookings/{bookingId}/billing`

```json
{ "action": "POST", "mode": "FULL" }
{ "action": "POST", "mode": "CUSTOM", "amount": 40, "reason": "No-show fee", "description":
{ "action": "WAIVE", "reason": "Included in the package" }
{ "action": "RESUME" }
```

`billing.status` is one of `NOT_BILLED` `PENDING` (Night Audit found no open folio) `POSTED` `VOIDED` `WAIVED` `NON_BILLABLE`. Voiding a posted charge is a folio correction made at the property — API keys are refused it (`403`).

Errors you will meet here

Status	Code	Meaning
400	`VALIDATION`	A field is wrong; details maps fields to messages.
400	`INVALID_REFERENCE`	An id that isn't this property's (route, provider, charge code…).
400	`DIRECTION_MISMATCH`	A drop-off on a pickup departure, or a route for the other direction.
400	`INVALID_STATUS_CHANGE`	That status can't follow the current one.
400	`PROVIDER_REQUIRED`	The property requires a provider before Assigned / Confirmed.

Status	Code	Meaning
400	`ALREADY_POSTED`	The transfer is already charged.
400	`NO_PRICE`	Full rate asked for a complimentary transfer — post a custom amount.
403	`SERVER_KEY_REQUIRED`	The key has browser origins.
404	`NOT_FOUND`	No such booking, departure or row at this property.
409	`IN_USE`	Delete refused — deactivate instead.
409	`DUPLICATE`	That code is already used at this property.
409	`MODULE_NOT_ENABLED`	The enterprise has no Transportation add-on (details.reason ADDON_NOT_ENABLED), or the property has it switched off (NOT_ENABLED).
409	`NO_OPEN_FOLIO`	The reservation has no open bill to post to.

Managing excursion and spa bookings

A "my booking" page for guests: the live status of a booking, and cancelling it while it is still free to cancel.

Look up a booking

`GET` `/activity-bookings/{reference}?email=ada@example.com`

Returns the same `booking` object the booking call returned, with its **live** status. Only bookings made with your key are visible, and only with the email they were made with (case doesn't matter). A wrong email reads exactly like an unknown reference: `404 BOOKING_NOT_FOUND`.

Status	Meaning
`CONFIRMED`	Booked and on.
`CHECKED_IN`	Spa only — the guest has arrived at the spa.
`IN_TREATMENT`	Spa only — the treatment has started.
`COMPLETED`	Done.
`NO_SHOW`	The guest didn't come.
`CANCELLED`	Cancelled — by the guest, or by the property (for example a departure called off for weather).

Moved excursions. If the property moves the guest to another departure (typically after cancelling the original for weather), the lookup follows it: `moved` is `true` and `departure` is the new one. Tell the guest the new date and time.

`cancellation.allowed` says whether the guest can still cancel online, and `cancellation.freeUntil` until when. Show a cancel button only when it is allowed.

Cancel a booking

`POST` `/activity-bookings/{reference}/cancel` server-only key

```json
{ "email": "ada@example.com", "reason": "Change of plans" }
```

Cancels the booking and takes the charge off the guest's bill. Allowed until the free-cancellation deadline — for an excursion, the excursion's own cutoff; for a treatment, the spa's cancellation window. After it, `409 CANCEL_CUTOFF_PASSED`: the guest must contact the property.

```json
{
  "booking": {
    "reference": "EXC-7K3QX9MD", "status": "CANCELLED",
    "payment": { "status": "PAID", "reference": "pay_123" },
    "cancellation": { "allowed": false, "freeUntil": "2026-10-03T05:00:00.000Z", "cancelledA
    "…": "…"
  }
}
```

> **REFUNDS ARE YOURS**
>
> When `refundRequired` is true the booking was sent as `PAID`: refund the guest through your own payment provider. The property sees the cancellation and your payment reference, and records the refund on its side.

Refusal	Meaning
`CANCEL_CUTOFF_PASSED`	Too late to cancel online — the guest contacts the property.
`ALREADY_CANCELLED`	It is already cancelled, completed or a no-show.
`BOOKING_NOT_FOUND`	No booking with that reference and email under your key.

Room bookings are looked up with `GET /bookings/{confirmationNo}` (see Rooms) and are cancelled through the property.

Webhooks

Be told when an excursion or spa booking your site made changes — without polling.

Why

After a guest books, the property may change the booking: call a boat trip off for weather, move the guest to another departure, mark a no-show, complete a treatment. A webhook posts each change to your server so you can email the guest or update your records straight away. You can always check a booking with the lookup endpoint as well.

Setting one up

The property adds your webhook URL to your API key in its administration area, chooses which events to send, and gives you the **signing secret** (it starts with `whsec_` and is shown to them once). Keep it with your API key as a server-side secret. The URL must be `https` and reachable from the internet.

Events

Event	When
`booking.confirmed`	A booking made with your key is confirmed.
`booking.cancelled`	The booking is cancelled — by the guest online, at the desk, or because the whole departure was cancelled.
`booking.moved`	Excursions — the property moved the guest to another departure.
`booking.completed`	Spa — the treatment is completed.
`booking.no_show`	The guest didn't come.
`ping`	A test the property sends from its administration area.

Only bookings made with your key are ever sent to your endpoints.

The request

```http
POST /webhooks/uppsolut HTTP/1.1
Content-Type: application/json
User-Agent: Uppsolut-Stay-Webhooks/1
Uppsolut-Webhook-Id: 5c0e1a2b-…
Uppsolut-Webhook-Event: booking.cancelled
Uppsolut-Webhook-Timestamp: 1790000000
Uppsolut-Webhook-Signature: v1=4f1c…

{
  "id": "5c0e1a2b-…",
  "event": "booking.cancelled",
  "createdAt": "2026-10-02T06:15:00.000Z",
  "data": {
    "booking": { "reference": "EXC-7K3QX9MD", "status": "CANCELLED", "…": "the same object t
  }
}
```

Verifying the signature

`Uppsolut-Webhook-Signature` is `v1=` followed by the hex HMAC-SHA256 of `{timestamp}.{raw body}`, keyed with your signing secret. Compute it over the **raw** request body — before any JSON parsing — compare in constant time, and reject messages whose timestamp is more than five minutes old.

javascript (node, express)

```
import crypto from "node:crypto";
import express from "express";

const SECRET = process.env.UPPSOLUT_WEBHOOK_SECRET; // whsec_…
const app = express();

app.post("/webhooks/uppsolut", express.raw({ type: "application/json" }), (req, res) => {
  const ts = req.header("Uppsolut-Webhook-Timestamp");
  const sig = req.header("Uppsolut-Webhook-Signature") ?? "";
  const expected = "v1=" + crypto.createHmac("sha256", SECRET).update(`${ts}.${req.body}`).d
  const fresh = Math.abs(Date.now() / 1000 - Number(ts)) < 300;
  const valid = sig.length === expected.length && crypto.timingSafeEqual(Buffer.from(sig), B
  if (!fresh || !valid) return res.sendStatus(400);

  const event = JSON.parse(req.body.toString("utf8"));
  // Deliveries are at-least-once: skip an id you have already handled.
  handle(event); // e.g. email the guest about a cancelled trip
  res.sendStatus(204);
});
```

Delivery and retries

Answer with any `2xx` within 10 seconds. Do the slow work after answering.

Anything else — another status, a timeout, a redirect — is retried: after about 1 minute, 5 minutes, 30 minutes, 2 hours, 6 hours and 12 hours, then given up.

Delivery is **at least once**: the same event can arrive twice. Use `id` to ignore repeats.

Events can arrive out of order after retries. The `booking` in each is a snapshot; for the current state, look the booking up.

> **ROTATING THE SECRET**
> When the property rotates the signing secret, messages are signed with the new one straight away. Update your server promptly; until then its signature check fails and the messages are retried.

Error codes

Every error is JSON with a stable code. Switch on the code, show your own message to the guest, and log the rest.

```json
{ "error": "Invalid request.", "code": "VALIDATION", "details": { "guest.email": "A valid em
```

Access

Status	Code	Meaning and what to do
401	`MISSING_API_KEY`	No key header. Fix the integration.
401	`INVALID_API_KEY`	Unknown, revoked or expired key. Ask the property for a new one.
403	`SCOPE_NOT_GRANTED`	The key isn't enabled for this module. Ask the property to tick it on your key.
403	`SERVER_KEY_REQUIRED`	Excursion and spa writes, and anything in Transportation, need a server-only key (no browser origins).
404	`PROPERTY_NOT_FOUND`	Unknown property, or one your key doesn't cover.
409	`MODULE_NOT_ENABLED`	The property isn't selling this module online right now (details.reason).
409	`MODULE_NOT_ENABLED`	Transportation — the enterprise has no Transportation add-on (details.reason ADDON_NOT_ENABLED) or the property has not switched it on (NOT_ENABLED).
429	`RATE_LIMITED`	Too many requests. Wait Retry-After seconds.
429	`TOO_MANY_HOLDS`	Your key has too many live holds. Book or let some expire.

Requests

Status	Code	Meaning and what to do
400	`VALIDATION`	Body or query failed validation. Map details onto your form fields.
400	`IDEMPOTENCY_KEY_REQUIRED`	Excursion/spa bookings need an Idempotency-Key header.
400	`INVALID_DATES`	Dates not YYYY-MM-DD, or in the wrong order.
400	`ARRIVAL_IN_PAST`	Before the property's business date. Build pickers off businessDate.
400	`TOO_FAR_AHEAD`	Rooms — beyond the property's booking window.
400	`MIN_STAY`	Rooms — fewer nights than the minimum.
400	`STAY_TOO_LONG`	The date range is too long for one call (62 nights / 62 days / 31 days).
400	`INVALID_OCCUPANCY`	Rooms — no adults, or more guests than the room type holds.
400	`INVALID_PARTY`	Excursions — at least one adult or child is needed.
400	`PARTY_TOO_LARGE`	More guests than allowed per online booking / per treatment.
404	`ROOM_TYPE_NOT_FOUND`	Not a sellable room type of this property.
404	`DEPARTURE_NOT_FOUND`	Not a departure of a published excursion at this property.
404	`TREATMENT_NOT_FOUND`	Not a published treatment at this property.
404	`HOLD_NOT_FOUND`	No such hold under your key.

Status	Code	Meaning and what to do
404	`BOOKING_NOT_FOUND`	Lookup: no match for that reference and email under your key.
400/409	`MEAL_PLAN_NOT_FOUND` `MEAL_PLAN_NOT_OFFERED` `ADD_ON_NOT_FOUND` `ADD_ONS_NOT_OFFERED`	Rooms — a meal plan or extra the property doesn't sell online.

Booking refusals

Status	Code	Meaning and what to do
409	`SOLD_OUT`	No room / not enough seats left. Send the guest back a step.
409	`SLOT_UNAVAILABLE`	Spa — the time is no longer free.
409	`STOP_SALE`	Rooms — the property closed one of the dates.
409	`NO_RATE`	No price for a night or date. The property must fill it in.
409	`BOOKING_DISABLED`	Rooms — online booking is off or not set up.
409	`DEPARTURE_CLOSED`	The departure is cancelled or has left.
409	`BOOKING_CUTOFF`	Too close to the start to book online.
409	`HOLD_EXPIRED`	The hold ran out. Start again from availability.
409	`HOLD_USED`	The hold was already booked.
409	`PRICE_CHANGED`	The price moved since you showed it (details.total). Re-quote and confirm.
409	`PAYMENT_NOT_CONFIGURED`	The property can't receive PAID bookings yet — send UNPAID.
409	`IDEMPOTENCY_CONFLICT`	That Idempotency-Key was used for a different request.
409	`ALREADY_CANCELLED`	Cancel: already cancelled, completed or a no-show.
409	`CANCEL_CUTOFF_PASSED`	Cancel: past the free-cancellation deadline.
400	`BOOKING_REJECTED`	Rooms — refused for another reason; error explains.

Status	Code	Meaning and what to do
500	`INTERNAL_ERROR`	Our side. Safe to retry a GET; retry a booking with the same Idempotency-Key.

Transportation has a few codes of its own (`INVALID_REFERENCE`, `IN_USE`, `DUPLICATE`, `DIRECTION_MISMATCH`, `ALREADY_POSTED`…) — see Transportation.

Go-live checklist

Work through this together — the property's administrator and the web developer — before the booking pages go public.

The property

- The key covers the site's property — or all of the group's properties, for a group portal — and is ticked for exactly the modules the site sells.
- The key has no browser origins (server-to-server), unless the site is static and books rooms only.
- Rooms: a rate plan chosen, meal plan, minimum stay, booking window and desk note set; the price calendar is filled across the booking window.
- Excursions: selling online switched on; the excursions to sell published with a description and photos; departures scheduled; prices set for the dates.
- Spa: selling online switched on; treatments published; therapists' working hours and treatment rooms set up; prices set.
- If the site takes payment: a payment method chosen for paid online bookings, per module.
- Policies written the way the desk will honour them.
- Someone owns key rotation and knows where to revoke.

The website

- The key (and any webhook secret) is stored as a server secret — not in any browser bundle or repository.
- Sections render from `modules`; nothing is hard-coded that the property can switch off.
- Date pickers start at `businessDate`.
- The price shown is always a quote's `grandTotal`, labelled according to `pricesIncludeTaxes`.
- If you take payment: hold → pay → book with the `holdId`, and send `expectedTotal`.
- Every booking call sends a fresh `Idempotency-Key`, and a retry after a timeout reuses it.
- `SOLD_OUT`, `SLOT_UNAVAILABLE`, `HOLD_EXPIRED`, `PRICE_CHANGED` and `BOOKING_CUTOFF` all have a guest-friendly message and a way back.

The confirmation page and email show the reference, date, time (and meeting point), total, and the policies.

"My booking" uses reference + email, and offers cancel only while `cancellation.allowed`.

Refunds happen in your payment provider when `refundRequired` is true.

If you use webhooks: signatures are verified on the raw body, and repeats are ignored by `id`.

Test together

A test room booking is found at the front desk with the "Booked via website" remark.

A test excursion booking is on the departure's manifest marked Online, with its reference; a paid one shows a settled bill.

A test spa booking is on the spa schedule marked Online; a hold shows as "Held online" until it expires or is booked.

Cancel one at the desk: the lookup shows it cancelled, and the webhook arrives.

Cancel one from the site: the charge comes off the bill.

Fill a departure: the site shows it full and refuses a booking.

The Online bookings list in the property's administration area shows all of the above, including any refused attempts.

Changelog

Everything here is additive: nothing is removed or renamed within v1.

October 2026 — stop sale documented

New Stop sale section on the Rooms page: how a closed night appears in availability, the quote and the booking refusal, and that a website can never book through one. No change to the API itself.

October 2026 — Transportation

Transportation (new scope `TRANSPORT`): the property's transfer set-up, transfer bookings, shared departures, the daily board and report data, and posting or waiving transfer charges, under `/properties/{id}/transport`. Server-to-server keys only. See Transportation.

The API now also answers `PATCH` and `DELETE` (Transportation only). Existing endpoints are unchanged.

The Transportation booking list is the first cursor-paginated list: `limit` and `cursor` in, `nextCursor` out.

September 2026 — documentation: new address, dates for Excursions and Spa

This guide moved from `/docs/api-integration` to `/docs/api`. Old links redirect permanently, so nothing breaks, but update your bookmarks. No change to the API itself.

Clarified what "today" means. Rooms keep using the property's `businessDate`; Excursions and Spa run on the actual date in the property's time zone, so activity date pickers should start at whichever of the two is later. See Conventions. No change to the API itself.

September 2026 — Excursions, Spa, webhooks

Excursions: catalogue, departures with live seats, quotes, holds, instant bookings.

Spa: treatments, free times with optional therapist gender, quotes, holds, instant bookings for one guest or a couple.

Managing bookings: `GET /activity-bookings/{reference}` and guest self-cancel.

Webhooks: signed notifications when the property changes a booking.

Scopes: keys are enabled for `ROOMS`, `EXCURSIONS` and/or `SPA`. Existing keys are `ROOMS` keys. New `403 SCOPE_NOT_GRANTED`.

Modules: `GET /properties/{id}` gains a `modules` block.

Rate limits: per-key limits, `429 RATE_LIMITED` with `RateLimit-*` and `Retry-After` headers.

September 2026 — meal plans and extras

Rooms: guests may choose a meal plan and add paid extras where the property offers them.

September 2026 — first release

Rooms: properties, property details, availability, quote, bookings, booking lookup.